



TEORÍA DE ALGORITMOS
CURSO ECHEVARRIA

Trabajo Práctico 1

May 31, 2025

- Juan Claros - 111254
- Yoel Gaston Arlia - 111520
- Guido Cuppari - 111172
- Josue Daniel Mamani - 111514

Índice

1	<u>Problema 1</u>	3
1.1	Enunciado y Supuestos	3
1.1.1	Enunciado	3
1.1.2	Supuestos	3
1.2	Diseño	3
1.3	Pseudocódigo	4
1.4	Análisis de Complejidad	5
1.5	Seguimiento	6
1.6	Casos de prueba y mediciones	8
1.7	Resultados	8
1.8	Conclusiones	9
1.9	Generación de sets	9

1 ***Problema 1***

1.1 Enunciado y Supuestos

1.1.1 Enunciado

- Cualquier cadena puede ser descompuesta en secuencias de palíndromos. Por ejemplo, la cadena ARACALACANA se puede descomponer de las siguientes formas: ARA CALAC ANA
ARA C ALA C ANA
A R A CALAC A N A
etc.

Desarrollar un algoritmo de programación dinámica que encuentre el menor número de palíndromos que forman una cadena dada. Por ejemplo, para ARACALACANA debería devolver 3.

1.1.2 Supuestos

- Se supone que el algoritmo recibe una cadena (string), con una frase, y el algoritmo buscara la mínima cantidad de palíndromos posibles que forma la cadena dada por parámetro, mediante programación dinámica.
- La frase, si tiene espacios o caracteres especiales, van a ser tratados como letras, ejemplo A!NEUQUEN!A, va a devolver 1, ya que se es indiferente con los caracteres y letras.

1.2 Diseño

- La ecuación de recurrencia a la que se se llega es: $opt[i] = 1$ si $palabra[i:]$ es palíndromo y $opt[i] = \min(opt[j+1] + 1)$ para todo j tal que $palabra[i:j+1]$ es palíndromo.
- Subestructura óptima: La solución óptima para el prefijo $palabra[0:i]$ depende de las soluciones óptimas de los sufijos $palabra[j+1:]$, que son subproblemas del problema original.
Subproblemas superpuestos: Evaluar si una subcadena es palíndromo se repite muchas veces. Por eso se usa una tabla booleana `es_palindromo[i][j]` para memorizar los resultados.
- Se usa memoization en 2 estructuras de datos, para guardar en un arreglo de enteros (todos infinitos positivos de base) $opt[i]$ el mínimo numero de palíndromos para $palabra[i:]$. Y para guardar en una matriz booleana (con todas las posiciones false de base) de $n \times n$ `es_palindromo[i][j]` si $palabra[i:j+1]$ es un palíndromo.
- Las estructuras de datos utilizadas para esta resolución son una lista de enteros, y una matriz de booleanos, siendo `opt` la lista y `es_palindromo` la matriz.

1.3 Pseudocódigo

```
1 FUNCION min_palindromos(palabra):  
2   n = largo(palabra)  
3  
4   optimo = [float('infinito')] * (n + 1)  
5   opt[n] = 0  
6  
7   es_palindromo = [[False] * n for _ in range(n)]  
8  
9   Para i desde n - 1 hasta 0 con paso -1 hacer  
10     Para j desde i hasta n - 1 hacer  
11       Si palabra[i] = palabra[j] y (j - i <= 1 o  
12       es_palindromo[i + 1][j - 1]) entonces  
13         es_palindromo[i][j] <- Verdadero  
14       FinSi  
15     FinPara  
16   FinPara  
17  
18   Para i desde n - 1 hasta 0 con paso -1 hacer  
19     Para j desde i hasta n - 1 hacer  
20       Si es_palindromo[i][j] entonces  
21         opt[i] <- minimo entre opt[i] y (1 + opt[j +  
22         1])  
23       FinSi  
24     FinPara  
25   FinPara  
26  
27   DEVOLVER opt[0]  
28  
29 cadena = "ALLAERRESAGASRADAR"  
30 print(min_palindromos(cadena))
```

1.4 Análisis de Complejidad

- Los puntos clave para analizar la complejidad de este algoritmo son los 2 for anidados que hay tanto para el procesamiento de los palíndromos que se ejecuta para cada par i, j en:

```
1     for i in range(n - 1, -1, -1):
2         for j in range(i, n):
3             if palabra[i] == palabra[j] and (j - i <= 1 or
es_palindromo[i + 1][j - 1]):
4                 es_palindromo[i][j] = True
5
```

Y para la construcción de la tabla booleana $opt[i][j]$ que también revisa cada subcadena $palabra[i : j + 1]$, y si la misma es palíndromo realiza una operación constante en:

```
1     for i in range(n - 1, -1, -1):
2         for j in range(i, n):
3             if es_palindromo[i][j]:
4                 opt[i] = min(opt[i], 1 + opt[j + 1])
5
```

Por lo tanto, en ambos for anidados, el peor caso implica una doble iteración sobre n , por ende la complejidad temporal total del algoritmo es de $O(N^2)$, siendo N el largo de la cadena.

1.5 Seguimiento

- Vamos a hacer el seguimiento para el caso de la cadena "ARACALACANA": lo primero que hace el algoritmo es obtener el largo de la cadena recibida mediante `len()`, normalizar todo a mayúsculas y procede a generar las 2 estructuras utilizadas, la lista de enteros y la matriz de booleanos, para luego llegar a la primer parte importante, el preprocesamiento de los , donde se marca como True toda subcadena de `palabra[i : j + 1]` que sea un palíndromo.

`n = largo(palabra) = 11`

`opt = [inf, inf, ..., inf, 0]` (longitud 12)

`es_palindromo[i][j] = False` para toda i, j

- Para ver si un substring es palíndromo o no, se utilizan 3 factores, si `palabra[i] = palabra[j]` (la palabra empieza y termina con la misma letra) y si $j - i \leq 1$ (es de 1 o 2 caracteres) o `es_palindromo[i+1][j-1] = True`. Entonces empieza desde $i = 10$ y siempre lo primero que llena en cada i es la diagonal, que es trivialmente un palíndromo, al ser un solo carácter, pero empieza llenando la matriz desde el final.
- La primer posición analizada es `[10][10]`, siendo la cadena "A", que es un caso trivial, cumpliendo la condición y llenando la tabla en esa posición con True, luego sigue `[9][9]`, que ocurre lo mismo que en el caso anterior, siendo simplemente el carácter "N", `[9][10]` no cumple la condición, ya que no empieza y termina con la misma letra, quedándose en False..., saltamos al ejemplo del siguiente palíndromo no trivial, que es `[8][10]` la cadena "ANA" cumple que empieza y termina con la misma letra, no tiene menos de 2 caracteres, pero `es_palindromo[i+1][j-1]` que en este caso es `[9][9]` es True, entonces rellena True dándola como palíndromo.
- Para el final de las iteraciones el algoritmo rellena la tabla ubicando True en todos los palíndromos que encuentra.

	A	R	A	C	A	L	A	C	A	N	A
A	T	F	T	F	F	F	F	F	F	F	F
R	F	T	F	F	F	F	F	F	F	F	F
A	F	F	T	F	T	F	F	F	T	F	F
C	F	F	F	T	F	F	F	T	F	F	F
A	F	F	F	F	T	F	T	F	F	F	F
L	F	F	F	F	F	T	F	F	F	F	F
A	F	F	F	F	F	F	T	F	T	F	F
C	F	F	F	F	F	F	F	T	F	F	F
A	F	F	F	F	F	F	F	F	T	F	T
N	F	F	F	F	F	F	F	F	F	T	F
A	F	F	F	F	F	F	F	F	F	F	T

Figure 1: Representación de la matriz de palíndromos.

- Una vez rellenada la tabla, pasa a la construcción de $\text{opt}[i]$ buscando la mínima cantidad de palíndromos de la palabra, donde $\text{opt}[n] = 0$, siendo el caso base la string vacía, luego empieza la iteración desde $i = 10$ hasta 0.
- Entonces empieza con $[10][10]$, le pregunta a la tabla si es palíndromo, en este caso lo es, por ende $\text{opt}[i]$ es igual al mínimo entre si mismo, que de base es infinito, y $1 + \text{opt}[j+1]$, que en este caso es $1 + \text{opt}[11]$ (que es 0), entonces $\text{opt}[10] = 1$. Sigue con $i = 9$, $j = 9$, se le pregunta si la $[9][9]$ es True, lo cual es correcto, entonces se usa la ecuación siendo $\text{opt}[9] = \min(\text{opt}[9], 1 + \text{opt}[10])$, que termina siendo $\text{opt}[9] = \min(\text{infinito}, 2)$, $\text{opt}[9] = 2$, y así sigue con toda la tabla para finalmente encontrar el optimo que es 3.

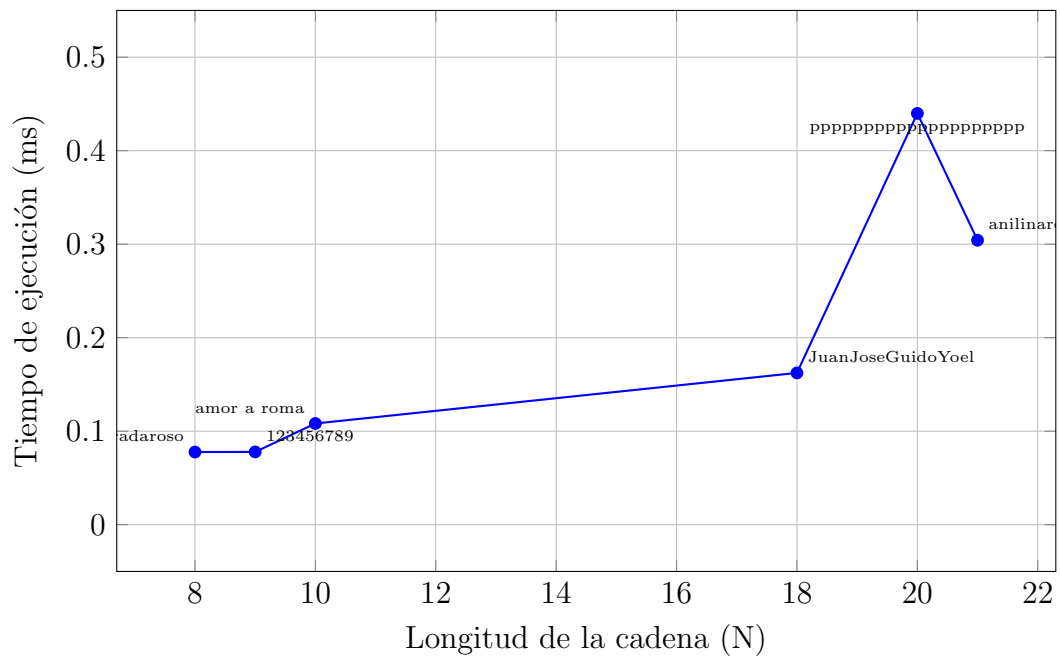
1.6 Casos de prueba y mediciones

Entrada	Cantidad de palíndromos	Tiempo (ms)
radaroso	2	0.077725
123456789	9	0.077863
amor a roma	1	0.108240
JuanJoseGuidoYoel	15	0.162336
pppppppppppppppppppppp	1	0.439882
anilinareconocersalas	3	0.304222

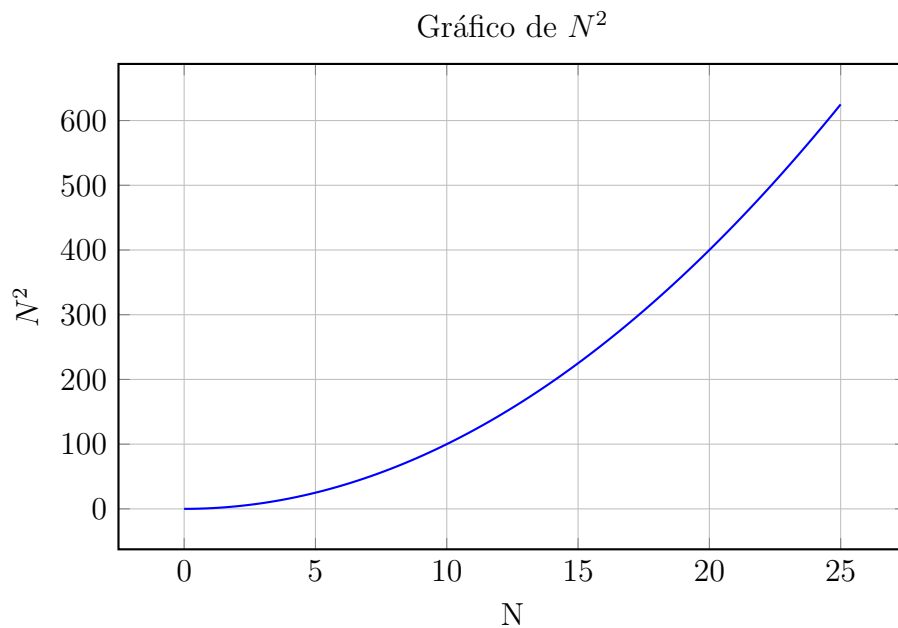
1.7 Resultados

Tiempo de ejecución vs longitud de la cadena (N)

Relación entre longitud N y tiempo de ejecución



Curva de N^2



1.8 Conclusiones

- Se ve que los sets cumplen con la complejidad prevista, exceptuando el caso de las p, donde se deja ver que las operaciones descartadas por ser menores a N^2 afectan a la complejidad, exceptuando este caso, se ve que a medida que aumenta el largo, el tiempo sube exponencialmente, cumpliendo lo previsto.

1.9 Generación de sets

- Los sets fueron generados arbitrariamente, para poder probar casos como distinción entre minúscula y mayúscula, espacios, ningún palíndromo, palíndromos encadenados, y una cadena de un solo carácter.